

OCC: SOFTWARE ENGINEER YEAR 2

SAQA ID

NQF 6

Software design and testing C#

SDT621

SUMMATIVE ASSESSMENT

2026



Table of Contents

DECLARATION OF AUTHENTICITY	3
QUALIFICATION AND ASSESSMENT DETAILS	4
Instructions:	5
Submission Requirements	5
Evidence Collection Requirements	5
To support your assessment submission, you must:	5
GitHub Submission Requirement	5
Summative Assessment Criteria:	6
Example of Collecting Evidence for your submission pdf:	6
SECTION A — Knowledge Questions (40 Marks)	7
True / False Questions [10]	10
SECTION B — Short Practical Questions (10 Marks)	11
Question 1	11
SECTION C: Scenario-Based Practical [50 Marks]	12
Question 1	12
SECTION C.2 — Button Event Logic Implementation (20 Marks)	12
SECTION C.3 — Software Testing Report (15 Marks)	13
SECTION C.4 — GitHub Version Control Evidence (5 Marks)	14
SECTION A — Knowledge Questions (40 Marks)	15
SECTION B — Short Practical Questions (10 Marks)	16
SECTION C — Scenario-Based Practical (50 Marks)	17
SECTION C.2 — Button Event Logic Implementation (20 Marks)	17
SECTION C.3 — Software Testing Report (15 Marks)	17
SECTION C.4 — GitHub Version Control Evidence (5 Marks)	18
SECTION C Mapping Summary	18

DECLARATION OF AUTHENTICITY

I (FULL NAME) Rhyno Botha

hereby declare that the contents of this assessment [SDT621]
are entirely my own work, completed by me without any paraphrasing/copying, or presented as my own
work not accessed from any **AI Apps, example ChatGPT, Co-pilot, Perplexity, or any other App.**

I have read and understood the assessment requirements and the Examination Rules and Regulations.

Signature: R.Botha

Date: 5/27/2026

QUALIFICATION AND ASSESSMENT DETAILS

Programme Title	OCC: SOFTWARE ENGINEER
Module Title	Software design and testing C#
Module Code	SDT621
NQF Level	6
Assessment Type	SUMMATIVE ASSESSMENT
Hand Out Date	27 / 0 5 / 2026
Hand in Date	27 / 05 / 2026
Total Marks	100
Assessment Lead Developer	Lusukama Selemani
Internal Moderator	Faith Muwishi

Student Name and Surname	Rhyno Botha
Student Number	20240091

Instructions:

Test or Debug Source Code to Ensure Client Needs Are Met

Please read the following instructions carefully before attempting this assessment:

- Read each question carefully and consider the mark allocation before answering.
- Answer all questions in the assessment.
- Provide clear, well-structured C# code and explanations where required.
- Apply your understanding of software testing and debugging techniques when solving the given scenarios.
- Ensure that your solutions meet the client requirements described in the scenario.
- Use meaningful variable names, proper formatting, and correct programming logic.
- Test your application before submission to confirm that it runs correctly.

Submission Requirements

- For your final submission, you must include:
- A signed Declaration of Authenticity (if not already included in the assessment document provided)
- Your completed Answer Sheet (PDF document) (preferably the official template provided with this assessment)
- Your Visual Studio project files (compressed as a ZIP file)

Evidence Collection Requirements

To support your assessment submission, you must:

- Take a screenshot of the program output for each question
- Ensure each screenshot shows the full screen, including the system date and time
- Ensure screenshots clearly demonstrate correct program execution
- Do NOT submit screenshots of your source code
- Submit your complete Visual Studio project folder as a ZIP file

GitHub Submission Requirement

You are required to:

- Push your project code to GitHub
- Ensure the repository contains the correct and complete solution
- Share the GitHub repository link in your submission for verification and review
- Example: <https://github.com/your-username/project-name>

Summative Assessment Criteria:

Applied Knowledge

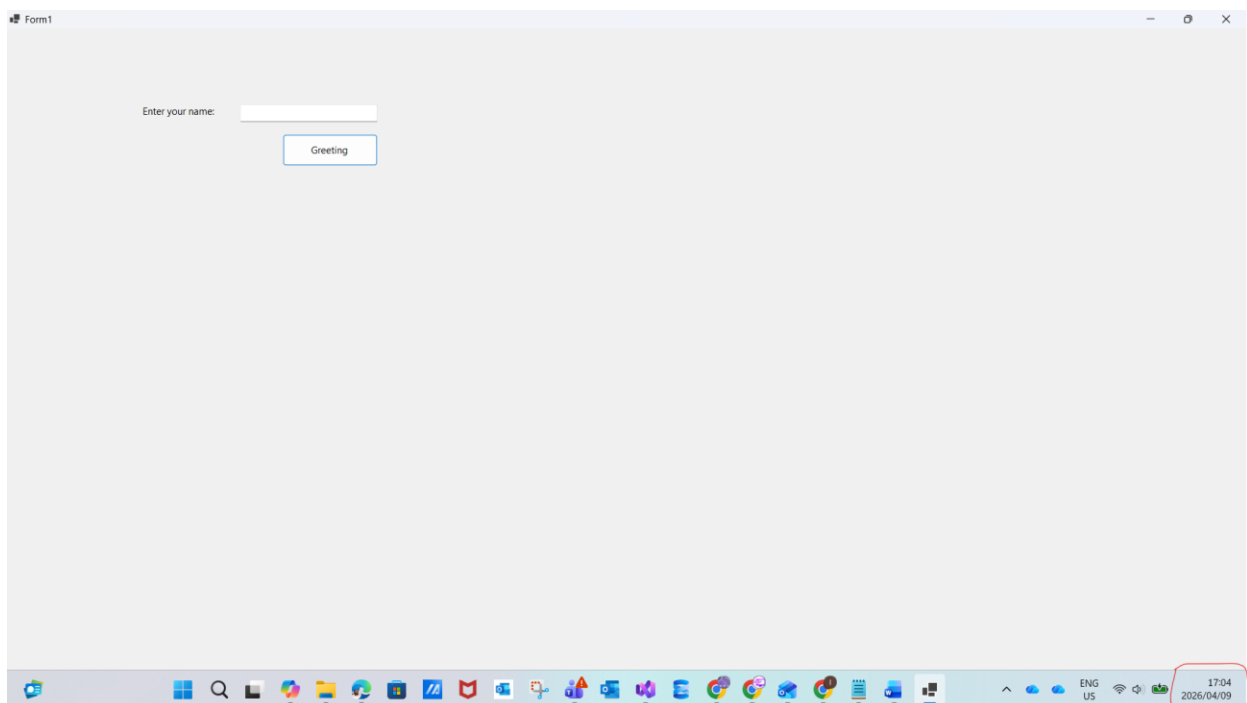
- AK0101 Knowledge of application testing techniques
- AK0102 Knowledge of test report writing

Internal Assessment Criteria

- IAC0101 functional and non-functional tests are identified
- IAC0102 The correct tests are applied
- IAC0103 A test report is produced

Example of Collecting Evidence for your submission pdf:

Question1 :



Ensure each screenshot shows the **full screen**, including the **system date and time**

SECTION A: Knowledge Questions (40 Marks)

Question 1

(30 Marks)

- 1.1 Define non-functional testing and explain what type of system qualities it evaluates. (2)

Non-Functional testing is testing the system / program outside of business requirements such as testing the overall performance and usability of the program

- 1.2 Explain load testing as a non-functional testing type. (1)

Load testing is when the developer prepares the infrastructure and program for a predetermined amount of concurrent users

- 1.3 Describe the difference between load testing and stress testing in non-functional test design. (2)

Load testing is when the developer knows the predetermined amount of concurrent users. Stress testing is when the opposite of load testing in the sense that the developer tests an unknown amount of concurrent users that is higher than the concurrent users in the load testing to see how the system will function under a lot of pressure from so many users.

- 1.4 Explain the difference between black-box testing and white-box testing. (3)

White-box testing is when the tester understands and tests the underlying infrastructure such as the code, the data structure used, if objects were used and if correct validation is used. Black-box testing is when the user doesn't understand the underlying infrastructure but they test the application to see how a standard user would use the program, by ensuring all the objects work on the form, correct error messages do display and doesn't close the program.

- 1.5 Describe equivalence partitioning and explain how it improves functional testing efficiency (3)

Equivalence partitioning is when we take a range of two numbers such as 10 and 50 and we identify two invalid scenarios and one valid scenario with those numbers, so in this case of "10 and 50", the equivalence partitioning will be 8, 46, 64. 8 is invalid since it's less than 10, 46 is valid since it's between 10 and 50 and 64 is invalid since it is greater than 50. It overall improves testing efficiency because the tester can understand and see the conditional statement used for "10 and 50" and if the tester types in the incorrect values like 8 and 64 then the user should be prompted with a message saying "these values are invalid" as an example.

- 1.6 A Loan Eligibility Processing System accepts customer income values between:

R1 000 and R150 000 Design TWO boundary value test cases to verify correct system behaviour. (4)

Lower boundary values : R999, R1000, R1001

Upper Boundary values : R149 999, R150 000, R150 001

- 1.7 Define error-case testing. (1)

Error-case testing is to test and examine what the system will do to evaluate purposeful errors, example : will the program crash from a Format Exception or will it be handled smoothly to show the user a Message indicating a descriptive message

- 1.8 Explain why testing unexpected user inputs is important when validating software reliability. (2)

To test if the software has correct error-logic and validation across input, the core logic of the system and if the system can display the user inputs. In the case of user inputs, can the system handle and display a message to the user if the incorrect data type was used or if a string had too many characters.

- 1.9 Provide TWO examples of error-case tests suitable for a login system. (2)

- 1) Error case 1: If a user signed up with different credentials and the user tries to login with not the same credentials, can the system give a message as a warning to the user or will the system freeze and crash.
- 2) Error case 2: In the case of a system where the username can only be a string, if a user types in a double or int instead, can the program handle this FormatException smoothly or will the program crash.

1.10 Define a test case in software testing. (2)

A test case is having predetermined test cases to examine how the program would handle these predetermined tests where the developer will have an expected outcome and the actual result will show if the program passed or failed the test.

1.11 List THREE components typically included in a structured test case. (3)

Test Case ID:
Description:
Expected Result:
Actual Outcome:
Status (Pass / Fail)

1.12A testing session executed:

Test Cases	Passed	Failed
100	80	20

Prepare a structured test summary section for a system testing report including:

Test results overview

Recommendation to stakeholders (5)

Test Results Overview :

Test Cases : 100

Passed : 80

Failed : 20

Recommendation to stake holders:

1) For the 20 test cases that failed, ensure those tests cases have been examined properly and executed accordingly to the expected result otherwise 20 test cases that failed can become a non-functional testing issue where the overall systems's performance gets affected drastically and the software becomes slow

2) Ensure proper validation is done across system by using a try..catch block, Try.Parse is used to check for conversion across the system, proper boolean flags is used to track when a statement became true or false and correct display of information to the user when an error was caught from the input

Total _____ / 30

True / False Questions

[10]

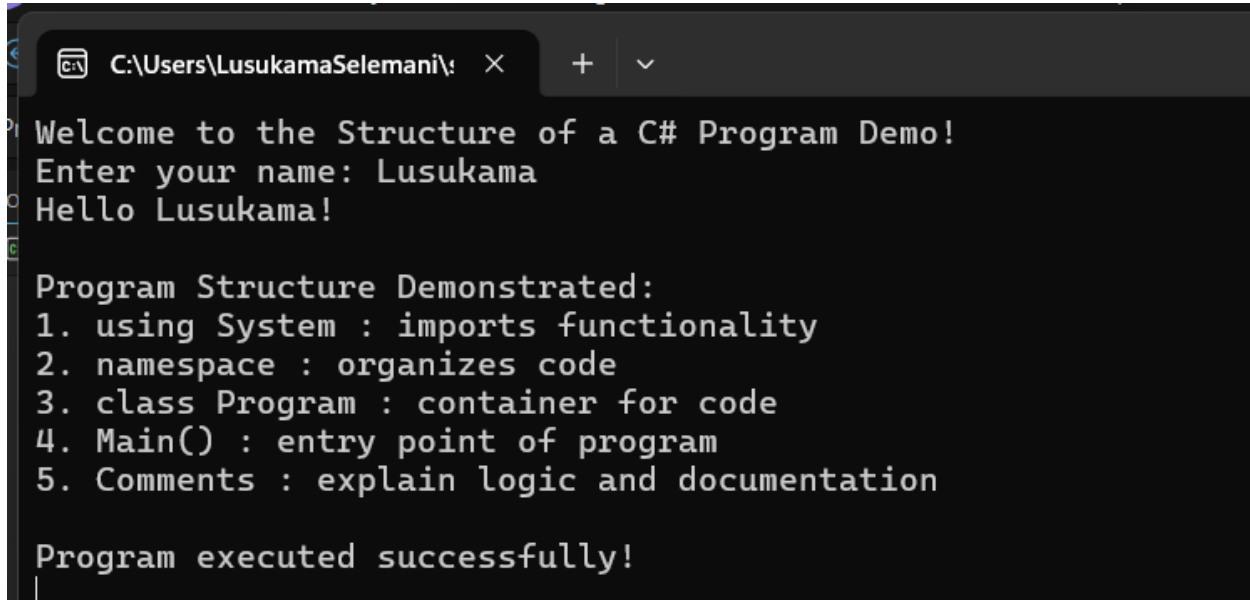
True / False Question	Answer
Exploratory testing allows testers to learn, design, and execute tests at the same time.	True
A good feature specification should be vague to allow flexibility during testing.	False
Specification review meetings help identify missing or incorrect requirements before testing begins.	True
Unit testing is typically performed after system testing.	False
Black-box testing focuses on testing system behavior without examining internal code structure.	True
White-box testing examines internal program logic and code structure.	True
Error-case testing helps ensure the system handles unexpected inputs correctly.	True
UX testing only evaluates system performance and ignores usability.	False
Security testing includes checking authentication and authorization mechanisms.	True
Maintainability testing helps determine how easily a system can be monitored and updated.	True

Total: ____ / 10

SECTION B: Short Practical Questions (10 Marks)

Question 1

1.1 Create a C# Console Application that the display the below output: (5)



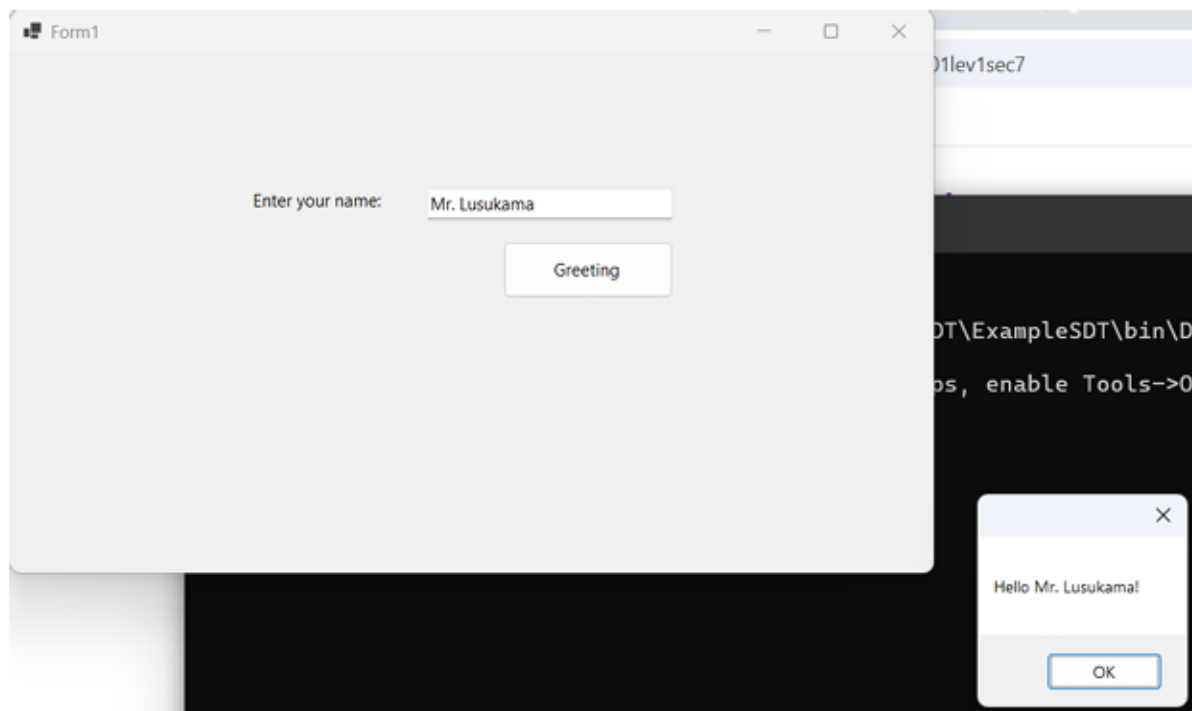
```

C:\Users\LusukamaSelemani\>
Welcome to the Structure of a C# Program Demo!
Enter your name: Lusukama
Hello Lusukama!

Program Structure Demonstrated:
1. using System : imports functionality
2. namespace : organizes code
3. class Program : container for code
4. Main() : entry point of program
5. Comments : explain logic and documentation

Program executed successfully!
  
```

1.2 Develop a Windows Forms Application that allows the user to enter their name and display a greeting message. (5)



SECTION C: Scenario-Based Practical [50 Marks]

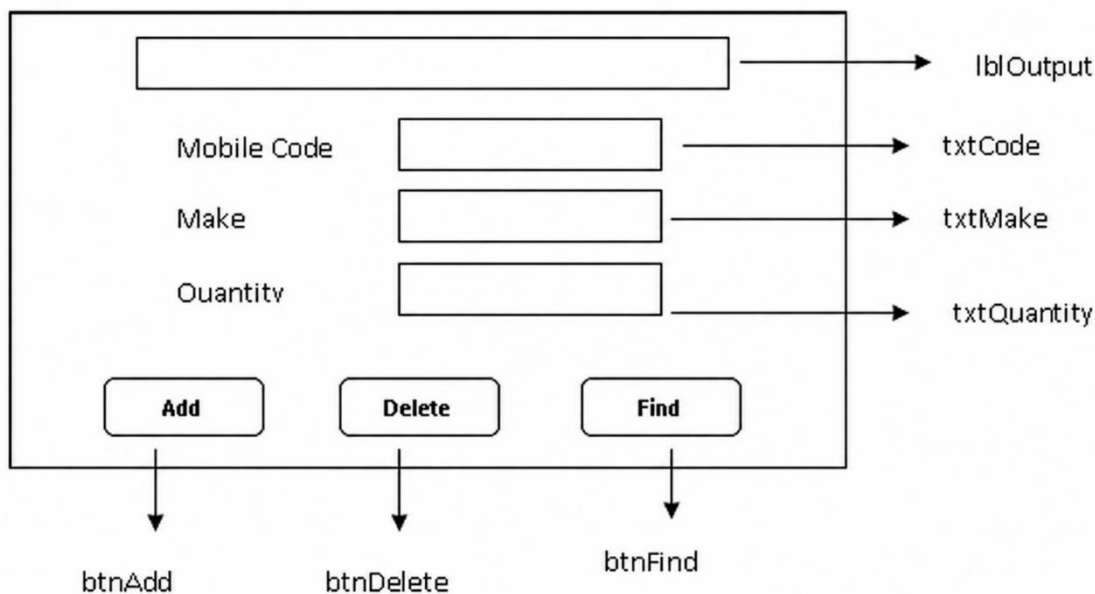
Question 1

You are working as a Junior Software Tester in a retail inventory company. The development team created a Mobile Stock Capture Windows Forms application, but the system contains interface naming issues, missing validations, and incorrect button behavior.

You are required to design, test, debug, and improve the Windows Forms application shown in the diagram.

SECTION C.1 — User Interface Design (10 Marks)

- 1.1 Create and design the following user interface using appropriate C# Windows Forms App controls and components. (10)



SECTION C.2 — Button Event Logic Implementation (20 Marks)

- 1.2 Write the events for each button

1.2.1 When the `btnAdd` button is clicked, the program should insert a record into the `tblMobilePhones` with the `MobileCode`, `Make` and `Quantity` from the corresponding data from the form and display the message "Record Added" in the `lblOutput`. (8)

1.2.2 When the `btnDelete` button is clicked, the program should delete the record whose `MobileCode` is typed in the `txtCode` text field and display the message "Record Found" in the `lblOutput`; or "Record NOT Found" if the record is not found. (7)

1.2.3 When the `btnFind` is clicked, the program should find the record whose

`MobileCode` is typed in the `txtCode` text field and display the message "Record Deleted" in the `lblOutput`. (5)

Note: use functional and non-functional tests for this question

SECTION C.3 : Software Testing Report (15 Marks)

1.3 Produce a Software Testing Report

Using the results from Question 1, compile a professional software testing report for the Mobile Stock Capture Application.

Your report must include the following sections:

1.3. 1 Application Description (3 Marks)

Provide a brief description of:

- Purpose of the application
- Main system functions
- User inputs captured

1.3.2 Test Cases Table (5 Marks)

Create a table similar to the example below:

Test Case	Input	Expected Result	Actual Result	Status

Include at least five test cases.

1.3.3 Identified Defects (4 Marks)

List any problems found during testing:

Defect Log:

1) DF-01:

Description: Typed in a value in the mobile code text box, clicked the delete button but when it was supposed to be deleted and prompted a message, instead it shows that the record was not found

Status : Open

Severity : Critical (The record is still added and shown in the table)

2) DF-02:

Description: Typed in a value in the mobile code text box, clicked on the search button but when it was supposed to be deleted, it shows that the record was not found

Status : Open

Severity : Critical (The record does exist in the table but it is not being found)

1.3.4. Recommendations for Improvement (3 Marks)

1) Instead of using raw data entires, initialize a List object that you can add values to it from the input of the user, following a foreach loop to read through the data entries in the list and then add them, additionally do this for the search and delete button to ensure records are properly being deleted or found. Ensure validation does exist across the entire system

- Extra recommendations :
- Use a try..catch block to handle incoming errors that can be handled smoothly additionally use the "clear()" method to enhance user usability.

SECTION C.4 — GitHub Version Control Evidence (5 Marks)

1.3.5 Use of GitHub

To obtain the full 5 marks, learners must demonstrate correct use of GitHub version control by completing the following tasks:

- Create and work within a separate branch (not directly on the main branch)
- Make meaningful and professional commit messages that clearly describe the changes made
- Successfully merge the branch into the main (origin/main) branch after completing development
- Ensure the repository contains the complete working solution
- Submit a valid and accessible GitHub repository link

End Exam

SECTION A — Knowledge Questions (40 Marks)

Purpose of Section A

Assess theoretical understanding of testing techniques, validation strategies, and structured test documentation required before practical debugging begins.

QCTO Mapping

Question Area	Marks	QCTO Criterion	Evidence Demonstrated
Non-functional testing definition	2	AK0101	Knowledge of testing concepts
Load testing explanation	1	AK0101	Understanding performance testing
Load vs Stress testing comparison	2	AK0101	Distinguishing test strategies
Black-box vs White-box testing	3	AK0101	Test method classification
Equivalence partitioning	3	AK0101	Functional test design
Boundary value test cases	4	AK0101	Boundary testing application
Error-case testing definition	1	AK0101	Exception scenario awareness
Unexpected input validation importance	2	AK0101	Reliability assurance concepts
Login error-case examples	2	AK0101	Functional validation thinking
Definition of test case	2	AK0102	Test documentation knowledge
Components of test case	3	AK0102	Structured testing documentation
Test summary & stakeholder recommendation	5	AK0102	Test reporting interpretation

Section A Mapping Summary

Criterion	Marks
AK0101 Knowledge of testing techniques	25
AK0102 Knowledge of test report writing	15

Total Section A = 40 Marks

SECTION B — Short Practical Questions (10 Marks)

Purpose of Section B

Assess ability to implement and validate basic working software functionality before applying structured testing.

QCTO Mapping Table

Task	Marks	QCTO Criterion	Evidence Demonstrated
Console output application	5	IAC0102	Correct functional output validation
Windows Forms greeting application	5	IAC0102	UI interaction testing

Section B Mapping Summary

Criterion	Marks
IAC0102 Correct tests applied	10

Total Section B = 10 Marks

SECTION C — Scenario-Based Practical (50 Marks)

Purpose of Section C

Assess full PM-04 competency:

Test or debug source code to ensure client needs are met

Learners must design, debug, validate, test, and document results.

SECTION C.1 — User Interface Design (10 Marks)

Task	Marks	QCTO Criterion	Evidence Demonstrated
Correct UI controls selected	2	IAC0101	Functional test readiness
Correct control naming	2	IAC0101	Interface clarity
Layout alignment with specification	3	IAC0101	UI correctness verification
Proper component usage	3	IAC0101	Interface usability validation

Mapping Summary

Criterion	Marks
IAC0101 Functional & non-functional tests identified	10

SECTION C.2 — Button Event Logic Implementation (20 Marks)

Task	Marks	QCTO Criterion	Evidence Demonstrated
Add button functionality	8	IAC0102	Record insertion testing
Delete button functionality	7	IAC0102	Conditional logic validation
Find button functionality	5	IAC0102	Search behaviour verification

Mapping Summary

Criterion	Marks
IAC0102 Correct tests applied	20

SECTION C.3 — Software Testing Report (15 Marks)

Task	Marks	QCTO Criterion	Evidence Demonstrated
Application description	3	AK0102	Documentation understanding
Test cases table	5	IAC0102	Test execution structure
Identified defects	4	IAC0101	Fault detection ability

Recommendations	3	IAC0103	Improvement reporting
-----------------	---	---------	-----------------------

SECTION C.4 — GitHub Version Control Evidence (5 Marks)

Task	Marks	QCTO Criterion	Evidence Demonstrated
Branch created and used	1	IAC0102	Version control workflow
Professional commit messages	1	IAC0102	Change tracking clarity
Multiple commits recorded	1	IAC0102	Development progression evidence
Branch merged into main	1	IAC0102	Integration workflow
Repository complete & accessible	1	IAC0103	Submission verification

SECTION C Mapping Summary

Criterion	Marks
IAC0101 Tests identified	14
IAC0102 Tests applied	26
IAC0103 Test report produced	10

Total Section C = 50 Marks

FINAL OVERALL PM-04 ASSESSMENT SUMMARY

QCTO Criterion	Description	Marks
AK0101	Knowledge of testing techniques	25
AK0102	Knowledge of test reporting	18
IAC0101	Functional & non-functional tests identified	14
IAC0102	Correct tests applied	36
IAC0103	Test report produced	7

TOTAL = 100 MARKS